

BLOGNEST

AI-POWERED BLOG MANAGEMENT API

PROJECT REPORT

Technology Stack
Node.js • Express.js • MongoDB • Mongoose
JWT • bcrypt.js • Google Gemini AI

Prepared as a professional project documentation report

Table of Contents

1. Project Overview	
2. Problem Statement	
3. Objectives	
4. Key Features	
5. Users and Responsibilities	
6. System Architecture	
7. MVC Structure	
8. Database Design	
9. Blog Lifecycle	
10. AI Features	
11. Security	
12. Software Requirements	
13. Hardware Requirements	
14. Project Setup	
15. Running and Testing the Backend	
16. API Testing Areas	
17. Expected Workflow	
18. Advantages of the System	
19. Project Conclusion	
20. Source Project References	

1. Project Overview

BlogNest is a backend REST API designed to make blog management simpler and more organized. The system provides the core services needed to create and manage blog posts, control user access, handle comments, organize content, and support AI-assisted writing. It is built with Node.js and Express.js, while MongoDB is used to store application data through Mongoose.

A major part of the project is the integration of Google Gemini AI. The AI layer supports blog generation and content summarization, allowing users to move from a basic topic or existing article to useful written content with less manual effort.

The backend is designed as a modular system. Authentication, authorization, business logic, database operations, and AI-related services are separated so that the application can be maintained and extended more easily.

2. Problem Statement

Managing a blog becomes difficult when posts, drafts, categories, tags, comments, users, and publishing activities are handled separately. The project addresses this by bringing the main content-management operations into one backend API.

- Managing multiple drafts, categories, and tags can become difficult.
- There is limited visibility into post performance and engagement.
- Content approval and comment moderation can take unnecessary time.
- Media paths and related metadata need a consistent place to be managed.
- Different users require different permissions for creating, reviewing, publishing, and reading content.

3. Objectives

- Provide a centralized REST API for blog management.
- Secure user access using JWT authentication and password hashing.
- Apply role-based permissions for Admin, Editor, Author, and Reader users.
- Support blog creation and management through CRUD operations.
- Organize posts using categories and tags.
- Provide controlled comment creation and moderation.
- Integrate AI features for blog generation and summarization.
- Keep the backend modular and suitable for future expansion.

4. Key Features

Authentication: JWT-based authentication with bcrypt.js password hashing.

Role-based access: Different permissions for Admin, Editor, Author, and Reader.

Blog management: Create, retrieve, update, and manage blog content and status.

Categories and tags: Organize content and make filtering and discovery easier.

Comments: Allow authenticated users to comment and provide moderation states such as approved, pending, and spam.

AI blog generation: Generate structured blog content through the AI service.

AI summarization: Create shorter summaries from longer article content.

Validation and sanitization: Validate request data and sanitize payloads to reduce common input risks.

Error handling and logging: Centralized handling and structured logging are part of the backend design.

5. Users and Responsibilities

Admin

Has the broadest level of access. The documented responsibilities include managing user registrations, changing roles, removing problematic comments, and inspecting logging information.

Editor

Works mainly with editorial activities, including reviewing pending posts, changing publishing status, and maintaining category information.

Author

Creates blog posts and works with content associated with the author's account.

Reader

Consumes published content and can create comments using an authenticated identity.

6. System Architecture

BlogNest follows a layered backend architecture. A request first enters the Express server, where routing and middleware are applied. Authentication and role checks protect restricted resources. Controllers then coordinate the application logic and communicate with the Mongoose data layer. AI-related requests are routed to the AI service when content generation or summarization is needed.

Main Components

API Interface: Tools such as Postman or Thunder Client send HTTP requests and receive JSON responses.

Express Server: Handles HTTP methods such as GET, POST, PUT, and DELETE, parses JSON requests, and manages routing.

Authentication and Authorization Middleware: Reads authentication tokens and checks whether the user's role is allowed to access a protected route.

Service / Logic Layer: Contains rules for blog lifecycles, comments, and reporting-related operations.

Database Layer: Uses Mongoose to work with MongoDB collections and schemas.

AI Flow: Connects the backend with AI functionality for blog generation and summarization.

7. MVC Structure

The project uses the Model-View-Controller pattern in a way that fits a headless REST API.

Model: Defines the data structure through Mongoose schemas. It controls fields, defaults, validation, and persistence in MongoDB.

Controller: Receives requests from routes, validates and processes the request, interacts with models, and returns JSON responses.

View / API Routing Layer: Because this is a backend API rather than a server-rendered website, there is no traditional HTML view. Routes act as the interface between HTTP requests and controller functions.

8. Database Design

MongoDB is the primary database. Mongoose is used as the ODM, providing schemas and validation before data is stored. The documentation identifies User, Post, and Comment relationships as the core data structure.

User Entity

- `_id` – ObjectId primary key
- `name` – required string
- `email` – required and unique string
- `password` – required hashed password
- `role` – user role, with reader as the documented default

Post Entity

- `_id` – ObjectId primary key
- `userID` – reference to the User collection
- `name` – required field representing the title
- `photo` – image/path information
- `message` – main blog content
- `likes` – numeric count with a default of 0
- `timestamps` – enabled for tracking record times

Relationships

- One User can have many Posts.
- One User can create many Comments.
- One Post can have many Comments.

9. Blog Lifecycle

The project describes a content workflow that can move a post through states such as Draft, Pending Approval, Scheduled, and Published. This separates content creation from editorial review and publication.

1. Draft
2. Pending Approval
3. Scheduled
4. Published

10. AI Features

AI is used as an additional service rather than replacing the normal blog-management workflow. The documented AI endpoints include `/api/ai/generate-blog` and `/api/ai/summarize`.

AI Blog Generation

The blog-generation service accepts a high-level topic or thematic input and uses an external language model to produce structured blog content. The project describes this as a way to speed up the drafting process.

Content Summarization

The summarization service processes longer article content and produces a concise summary that can be used as an abstract or supporting metadata.

11. Security

- JWT is used for stateless authentication.
- bcrypt.js is used for secure password hashing.
- Role-based route protection limits access to administrative and editorial operations.
- Request validation helps prevent invalid data from reaching the database.
- Input sanitization is included to reduce XSS and NoSQL injection risks.
- Rate limiting is identified as an additional security mechanism.
- Structured logging supports monitoring and troubleshooting.

12. Software Requirements

Operating System: Windows 10/11, macOS, or Linux

Node.js: Version 16 or above

npm: Version 8 or above

Express.js: Backend web framework

MongoDB: Primary database

Postman / Thunder Client: API testing

Visual Studio Code or similar: Development environment

13. Hardware Requirements

- Processor: Intel Core i5 8th Gen or above, AMD Ryzen 5, or equivalent.
- Memory: 8 GB minimum; 16 GB is recommended for concurrent services.
- Storage: At least 1 GB of available workspace.

14. Project Setup

The project setup described in the source document starts with a project folder and a Server folder. The Server directory is opened in VS Code and initialized as a Node.js project.

1. Create the project folder.
2. Create the Server folder and open it in VS Code.
3. Run `npm init -y` in the Server directory.
4. Create `server.js`.
5. Create the models, controllers, and routes folders.
6. Install Mongoose using `npm install mongoose`.
7. Configure the MongoDB connection and define the required schemas/models.

15. Running and Testing the Backend

After the database connection and backend configuration are in place, the documented workflow uses Postman or the Thunder Client VS Code extension for API testing.

1. Start the backend using npm start.
2. Send requests from Postman or Thunder Client.
3. Check authentication responses for registration and login.
4. Test blog creation and blog retrieval.
5. Test AI blog generation.
6. Test blog summarization.

16. API Testing Areas

User Registration: Verify that a new user can be registered with the required account information.

User Login: Verify authentication and token-based access.

Create Blogs: Verify that an authorized user can create blog content.

Fetch Blogs: Verify retrieval of blog records.

Generate AI Blogs: Verify the AI blog-generation flow.

Summarize Blogs: Verify that longer blog content can be summarized.

17. Expected Workflow

A typical request begins at the API endpoint and passes through the applicable middleware. For protected operations, the JWT is checked and the user's role is evaluated. The controller then performs the required operation through the appropriate model or service. MongoDB stores or retrieves the data, and the API returns the result as JSON.

For AI requests, the controller delegates the content task to the AI flow. The generated content or summary is then returned to the client through the API response.

18. Advantages of the System

- Centralizes blog-related backend operations.
- Separates responsibilities through a modular architecture.
- Provides controlled access based on user roles.
- Supports both traditional content management and AI-assisted writing.
- Uses MongoDB and Mongoose for structured document handling.
- Includes security measures such as authentication, validation, sanitization, and rate limiting.
- Can be consumed by different frontend or API clients because it is exposed as a REST API.

19. Project Conclusion

BlogNest combines the essential functions of a blog management system with AI-assisted content tools. Its backend is organized around Express.js, MongoDB, Mongoose, authentication middleware, controllers, and dedicated AI services. This structure makes it possible to manage users, blogs, comments, and content workflows from a single API while keeping access controlled by user roles.

The project also provides a practical foundation for further development. The modular structure can be extended with additional content-management features or AI capabilities without changing the basic purpose of the system.

20. Source Project References

The original project material includes references to a demo video, API testing video, and CodeDrive, but their actual links are not present in the parsed project document. They should be added here when the final URLs are available.

— *End of Project Report* —